

Containers and Kubernetes Lab Guide

One lab per video. Each builds on the one before it, from a single isolated process on your own Linux host to staged failures in a running cluster.

How to use this guide

Do the labs in order. Later labs assume the host, cluster, and habits from earlier ones. Type the commands rather than pasting them — typing is how the syntax sticks — and keep a plain-text notes file open for every lab.

If you are new to Linux, a few conventions appear on every page:

- Command blocks are typed into a terminal on your lab host, one line at a time. Press Enter after each line.
- Text in angle brackets, such as `<A-IP>`, is a placeholder. Replace the whole thing, brackets included, with your own value.
- `sudo` runs one command as the administrator. It will ask for your password the first time.
- `cat > file <<'EOF' ... EOF` writes everything between the two markers into a file. Paste the whole block at once.
- Lines starting with `#` are comments. You do not type them.
- `PID=$(...)` saves a command's output into a variable you can reuse as `$PID`. Variables vanish when you close the terminal — re-run the line if a later step fails.
- Commands ending in `-w` or `-f` keep watching. Press Ctrl+C to stop them.

Every lab ends with a cleanup block. Run it. Containers, images, and volumes accumulate quickly, and leftovers from one lab cause confusing failures in the next.

Lab setup reference

Lab 3 walks through building this environment. Use it as the checklist to confirm you have everything before starting a part.

Item	What to use	Needed from
Lab host	Ubuntu Server 24.04 LTS. 4 vCPU, 8 GB RAM, 40 GB disk. A VM you can snapshot is ideal.	Lab 1
Second host	A clone of the lab host on the same network.	Labs 9, 14
Container engine	Docker Engine (current stable), with the Compose and Buildx plugins it installs.	Lab 1
Host tools	jq, util-linux (nsenter, lsns, findmnt), iproute2, procs, libcap2-bin, tree	Lab 2
VM tool	Multipass (for the comparison in Lab 1 only)	Lab 1
Supply chain	trivy, syft, cosign	Lab 10
Kubernetes	kubectl and kind. minikube works too; commands differ only for cluster setup and ingress.	Lab 18
Packaging and GitOps	helm, Argo CD, a Git repository you can push to	Lab 24
Cloud (optional)	A small managed cluster: EKS, GKE, or AKS. Delete it after the lab.	Lab 24
Mesh	linkerd CLI	Lab 28

Mac and Windows users: Docker Desktop runs a hidden Linux VM, so namespace and cgroup commands in Labs 2 and 5 will not see your containers. Use a real Linux VM or WSL 2 for the whole course.

Foundations

What a container is, how the kernel makes one, and the lab and method you will use for the rest of the course.

Lab 01 Container vs. VM, Side by Side

Prerequisites Lab host from Lab 3 (or any Linux machine with Docker); Multipass or another VM tool.

STEPS

- 1 Time a container from nothing to a finished command.

```
docker pull alpine:3.20
time docker run --rm alpine:3.20 echo "hello from a container"
```

EXPECT real 0m0.4s (typically well under one second)

- 2 Time a VM doing the same job.

```
time multipass launch 24.04 --name vm1
time multipass exec vm1 -- echo "hello from a VM"
```

EXPECT Launch: tens of seconds. Exec after boot: fast, but only because the whole OS is already running.

- 3 Compare size on disk.

```
docker image ls alpine:3.20
multipass info vm1 | grep -i disk
```

EXPECT alpine $\approx$ 8 MB vs. VM disk measured in GB

- 4 Count what each is running. The container has one process; the VM has a full init system.

```
docker run --rm alpine:3.20 ps aux
multipass exec vm1 -- ps aux | wc -l
```

- 5 Check the kernel. The container reports the host kernel — it has none of its own.

```
uname -r
docker run --rm alpine:3.20 uname -r
multipass exec vm1 -- uname -r
```

EXPECT Lines 1 and 2 match. Line 3 differs.

- 6 Record your numbers in a table: startup, size, process count, kernel.

CHECKPOINT

1. Why does the container report the host's kernel version?
2. Which of your measurements would change if the container image were 1 GB? Which would not?
3. Name one thing the VM isolates that the container does not.

IF IT GOES WRONG

- “permission denied ... docker.sock”: your user is not in the docker group yet — see Lab 3, step 3.

Stretch	Repeat the timings with a 1 GB image (e.g. python:3.12) on a cold cache. Which number moved, and why did the VM's not?
----------------	--

Cleanup	<code>multipass delete --purge vm1</code>
----------------	---

Lab 02 Seeing Namespaces and Cgroups from the Host

Prerequisites Lab 1. sudo on the host.

STEPS

- 1 Start a container with a memory limit and find its host PID.

```
docker run -d --name ns-demo --memory 256m --hostname inside nginx:1.27
PID=$(docker inspect -f '{{.State.Pid}}' ns-demo); echo $PID
```

- 2 Compare its namespaces with your shell's. Different inode numbers mean a different namespace.

```
sudo ls -l /proc/$PID/ns
ls -l /proc/self/ns
sudo lsns -p $PID
```

- 3 Enter just the UTS and PID namespaces and look around.

```
sudo nsenter -t $PID -u hostname
sudo nsenter -t $PID -p -m ps aux
```

EXPECT inside
(only nginx processes are listed)

- 4 Find the container's cgroup and read the memory limit the kernel is enforcing.

```
cat /proc/$PID/cgroup
CG=$(cut -d: -f3 /proc/$PID/cgroup)
cat /sys/fs/cgroup$CG/memory.max
cat /sys/fs/cgroup$CG/memory.current
```

EXPECT 268435456 (256 MiB)

- 5 Decode the capabilities the process holds.

```
grep Cap /proc/$PID/status
capsh --decode=$(grep CapEff /proc/$PID/status | awk '{print $2}')
```

- 6 See the layered root filesystem mounted for it.

```
docker inspect -f '{{.GraphDriver.Data.MergedDir}}' ns-demo
mount | grep overlay | head -n 3
```

CHECKPOINT

1. Which namespace would you need to enter to see the container's network interfaces?
2. What would `memory.max` read if you had not passed `--memory`?
3. Is `CapEff` the same as full root? How can you tell?

IF IT GOES WRONG

- cgroup v1 hosts use `/sys/fs/cgroup/memory/.../memory.limit_in_bytes` instead. Check with: `stat -fc %T /sys/fs/cgroup` (`cgroup2fs = v2`).

Stretch	Start a container with <code>--pid=host</code> and compare <code>/proc/self/ns/pid</code> with the host's. What isolation did you just give away?
----------------	---

Cleanup	<code>docker rm -f ns-demo</code>
----------------	-----------------------------------

Lab 03 Building Your Container Lab

Prerequisites A spare machine, a VM (4 vCPU, 8 GB RAM, 40 GB disk recommended), or WSL 2.

STEPS

- 1 Install Ubuntu Server 24.04 LTS (or Debian 12). Never use a shared or production host for this course.

- 2 Install Docker Engine from the official repository and the tools used throughout.

```
curl -fsSL https://get.docker.com | sudo sh
sudo apt install -y jq util-linux iproute2 procps libcap2-bin tree
```

- 3 Let your user run Docker, then log out and back in.

```
sudo usermod -aG docker $USER
docker version
```

- 4 Optional but recommended: set up rootless mode alongside and learn to switch contexts.

```
sudo apt install -y uidmap
dockerd-rootless-setuptool.sh install
docker context ls
docker context use rootless
```

- 5 Verify with a real workload and inspect it from the host.

```
docker run -d --name first -p 8080:80 nginx:1.27
curl -s localhost:8080 | head -n 4
docker inspect first | jq '[0].State'
ps -ef | grep [n]ginx
```

- 6 Learn the reset. Run this at the end of any lab that leaves clutter.

```
docker system df
docker rm -f $(docker ps -aq) 2>/dev/null
docker system prune -a --volumes -f
```

- 7 Take a VM snapshot named “clean-lab” so you can always return to this state.

CHECKPOINT

1. Why does Docker Desktop on Mac or Windows still involve Linux?
2. What does rootless mode protect you from if a container escapes?
3. What does `docker system prune -a` remove that plain `prune` does not?

IF IT GOES WRONG

- Rootless: “newuidmap not found” means uidmap is missing. “ports below 1024” fail by design — use 8080.

Stretch	Rebuild the lab from your snapshot and time how long a full reset takes. Script steps 2–5 so you can do it in one command.
----------------	--

Cleanup	<code>docker rm -f first</code>
----------------	---------------------------------

Lab 04 Writing a Diagnostic Plan

Prerequisites Video 4. Paper or a text file.

SCENARIO

After Friday’s release, the “reports” container restarts about every 30 seconds. The image built cleanly in CI. The last log line before each restart is: `PermissionError: [Errno 13] Permission denied: '/data/cache'`. The same image ran fine on a developer laptop.

STEPS

- 1 Define the problem in one sentence: what fails, where, since when, and what still works.
- 2 List every layer — application, image, runtime, host, orchestrator — and write one hypothesis per layer.
- 3 Rank the layers. Pick the first to investigate and justify it from the evidence you already have.
- 4 Write the exact commands you would run to gather evidence, before it disappears.

```
docker ps -a --filter name=reports
docker inspect reports | jq '.[0].State, .[0].Config.User, .[0].Mounts'
docker logs --tail 50 reports
docker events --since 30m --filter container=reports
```

- 5 Write the one change you would test first, and what result would confirm or reject it.
- 6 Write what you will record in your notes regardless of the outcome.

CHECKPOINT

1. Why is “it works on my laptop” evidence rather than noise?
2. Which piece of evidence is lost if someone runs `docker rm reports`?

IF IT GOES WRONG

- If you cannot decide the first layer, choose the one that is cheapest to rule out.

Stretch Rewrite your plan assuming the container runs in Kubernetes instead. Which evidence commands change, and which layer moves up your ranking?

Lab 05 Mapping a Container from the Host

Prerequisites Labs 2–3.

STEPS

- 1 Start a container and locate it in the host process tree.

```
docker run -d --name map -p 8080:80 -e APP_MODE=lab nginx:1.27
ps -ef --forest | grep -B1 -A3 [c]ontainerd-shim
docker top map
PID=$(docker inspect -f '{{.State.Pid}}' map)
```

- 2 Compare users. Note the UID the host sees for each nginx process.

```
docker exec map id
ps -o user,uid,pid,cmd --ppid $PID -p $PID
```

- 3 Compare mounts.

```
docker exec map cat /proc/mounts | head
sudo findmnt -N $PID | head
```

- 4 Compare network: interfaces inside, the published port outside.

```
sudo nsenter -t $PID -n ip -br addr
ip -br link | grep veth
sudo ss -tlnp | grep 8080
```

- 5 Read the environment the runtime constructed.

```
sudo cat /proc/$PID/environ | tr '\0' '\n'
```

- 6 Send signals and watch how the container responds.

```
docker kill -s HUP map && docker logs --tail 3 map
time docker stop map
```

EXPECT nginx reloads on HUP; stop returns quickly because nginx handles SIGTERM.

- 7 Write a two-column table: “container sees” vs “host sees” for each item above.

CHECKPOINT

1. Why can the host see the container's processes but not the reverse?
2. What would make docker stop take the full 10 seconds?

IF IT GOES WRONG

- nsenter: "no such process" — the container stopped; restart it and re-read the PID.

Stretch	Run a container with --user 1000 and map its processes again. Then do the same in rootless mode and explain the UID the host shows.
----------------	---

Cleanup	<code>docker rm -f map</code>
----------------	-------------------------------

Images

What containers run from: layers, builds, optimization, distribution, and the supply chain behind every image.

Lab 06 Inspecting Layers, Tags, and Digests

Prerequisites Lab 3.

STEPS

- 1 Pull by tag and view the layer history.

```
docker pull nginx:1.27
docker history nginx:1.27
docker image inspect nginx:1.27 | jq '.[0].RootFS.Layers'
```

- 2 Find the immutable digest behind the tag and pull by it.

```
DIGEST=$(docker image inspect -f '{{index .RepoDigests 0}}' nginx:1.27); echo
$DIGEST
docker pull $DIGEST
```

EXPECT Pull completes instantly: every layer is already present.

- 3 Pull a related image and observe shared layers being skipped.

```
docker pull nginx:1.27-perl
```

EXPECT Several lines read “Already exists”.

- 4 See the writable layer at work.

```
docker run -d --name cow nginx:1.27
docker exec cow sh -c 'echo test > /usr/share/nginx/html/new.html'
docker diff cow
```

- 5 Measure size by layer. Install dive for an interactive view.

```
docker system df -v | head -n 20
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock wgoodman/dive
nginx:1.27
```

CHECKPOINT

1. If the nginx maintainers push a new 1.27 tomorrow, which of your two references changes?
2. Why does docker diff show C for some paths and A for others?

IF IT GOES WRONG

- “manifest unknown” on a digest pull usually means you copied a platform-specific digest for another architecture.

Stretch	Find the largest single layer in python:3.12 and name the instruction that created it.
----------------	--

Cleanup	<code>docker rm -f cow</code>
----------------	-------------------------------

Lab 07 From Naive to Multi-Stage Builds

Prerequisites Lab 6. Go is not needed on the host.

STEPS

- 1 Create a tiny Go web service.

```
mkdir -p ~/labs/l7 && cd ~/labs/l7
cat > main.go <<'EOF'
package main
import ("fmt"; "net/http")
func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "ok") })
    http.ListenAndServe(":8080", nil)
}
EOF
cat > go.mod <<'EOF'
module lab7
go 1.23
EOF
dd if=/dev/urandom of=junk.bin bs=1M count=200
```

- 2 Write the naive version.

```
cat > Dockerfile.naive <<'EOF'
FROM golang:1.23
WORKDIR /src
COPY . .
RUN go build -o /app .
CMD ["/app"]
EOF
time docker build -f Dockerfile.naive -t lab7:naive .
```

- 3 Change one line of main.go and rebuild. Note the time and that the build context includes junk.bin.
- 4 Write the refactored version and a .dockerignore.

Lab 08 Shrink, Harden, and Prove No Secrets Remain

Prerequisites Lab 7.

STEPS

- 1 Build the flawed starting image — the secret is added then “removed”.

```
mkdir -p ~/labs/l8 && cd ~/labs/l8
echo "TOKEN=s3cr3t-lab8" > .env
cat > Dockerfile.bad <<'EOF'
FROM python:3.12
COPY .env /tmp/.env
RUN pip install flask && rm /tmp/.env
CMD ["python", "-m", "flask", "--version"]
EOF
docker build -f Dockerfile.bad -t lab8:bad .
```

- 2 Recover the secret from the image layers.

```
docker save lab8:bad -o bad.tar && mkdir bad && tar -xf bad.tar -C bad
for l in bad/blobs/sha256/*; do tar -tf $l 2>/dev/null | grep -q tmp/.env && tar -
xOf $l tmp/.env; done
```

EXPECT TOKEN=s3cr3t-lab8

- 3 Rebuild properly: slim base, a build secret mount, labels, non-root user.

```
cat > Dockerfile <<'EOF'
# syntax=docker/dockerfile:1
FROM python:3.12-slim
LABEL org.opencontainers.image.title="lab8"
org.opencontainers.image.source="https://example.com/lab8"
RUN --mount=type=secret,id=token pip install --no-cache-dir flask
RUN useradd -u 10001 app
USER 10001
CMD ["python", "-m", "flask", "--version"]
EOF
docker build --secret id=token,src=.env -t lab8:good .
```

- 4 Repeat step 2 against lab8:good. It must print nothing.
- 5 Compare size, user, and labels.

```
docker image ls lab8
docker image inspect lab8:good | jq '.[0].Config.User, .[0].Config.Labels'
docker history --no-trunc lab8:good | grep -i token || echo "clean"
```

CHECKPOINT

1. Why did `rm` in the same RUN not help in the bad image?
2. Which belongs in a build argument and which at runtime: a package version, a database password?
3. What debugging tools did you lose by going slim?

IF IT GOES WRONG

- Older Docker: enable BuildKit with `DOCKER_BUILDKIT=1`.

Stretch	Shrink <code>lab8:good</code> further with a multi-stage build that copies only site-packages into a distroless Python base. Record the size.
----------------	---

Cleanup	<code>rm -rf bad bad.tar .env; docker image rm lab8:bad lab8:good</code>
----------------	--

Lab 09 Running a Private Registry

Prerequisites Two lab hosts (A and B) that can reach each other. Replace REG with host A's IP.

STEPS

- 1 On host A, create credentials and a self-signed certificate.

```
mkdir -p ~/reg/{auth,certs} && cd ~/reg
docker run --rm --entrypoint htpasswd httpd:2 -Bbn student labpass > auth/htpasswd
openssl req -newkey rsa:4096 -nodes -x509 -days 30 -keyout certs/reg.key -out
certs/reg.crt -subj "/CN=REG" -addext "subjectAltName=IP:REG"
```

- 2 Start the registry.

```
docker run -d --name registry -p 5000:5000 --restart unless-stopped -v
$PWD/auth:/auth -v $PWD/certs:/certs -v regdata:/var/lib/registry -e
REGISTRY_AUTH=htpasswd -e REGISTRY_AUTH_HTPASSWD_REALM=lab -e
REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd -e
REGISTRY_HTTP_TLS_CERTIFICATE=/certs/reg.crt -e REGISTRY_HTTP_TLS_KEY=/certs/reg.key
registry:2
```

- 3 Trust the certificate on both hosts (copy reg.crt to B).

```
sudo mkdir -p /etc/docker/certs.d/REG:5000
sudo cp reg.crt /etc/docker/certs.d/REG:5000/ca.crt
```

- 4 Tag, log in, push. Use a semantic version tag plus the git SHA — never only latest.

```
docker tag nginx:1.27 REG:5000/web:1.0.0
docker login REG:5000 -u student
docker push REG:5000/web:1.0.0
curl -s -u student:labpass --cacert certs/reg.crt https://REG:5000/v2/_catalog
```

- 5 On host B: pull without logging in (should fail), then log in and pull.

```
docker pull REG:5000/web:1.0.0
docker login REG:5000 -u student && docker pull REG:5000/web:1.0.0
```

EXPECT First pull: "no basic auth credentials". Second: success.

- 6 Promote without rebuilding: retag the same digest as :prod and push.

CHECKPOINT

1. How can you prove the :prod image is byte-identical to :1.0.0?
2. Which error would you see if the certificate were not trusted on B?

IF IT GOES WRONG

- “x509: certificate signed by unknown authority” → step 3 on that host. “toomanyrequests” on Docker Hub → rate limit; add a pull-through mirror.

Stretch	Configure host B’s daemon to use host A as a pull-through mirror for Docker Hub and confirm with docker info.
----------------	---

Cleanup	<code>docker rm -f registry; docker volume rm regdata</code>
----------------	--

Lab 10 Scan, Sign, SBOM, Verify

Prerequisites Lab 9 registry running. Install trivy, syft, and cosign.

STEPS

- 1 Scan a full image and a minimal one; compare the counts.

```
trivy image --severity HIGH,CRITICAL python:3.12
trivy image --severity HIGH,CRITICAL lab7:multi
```

- 2 Generate an SBOM and look inside.

```
syft REG:5000/web:1.0.0 -o spdx-json > web.sbom.json
jq '.packages | length' web.sbom.json
jq -r '.packages[].name' web.sbom.json | head
```

- 3 Generate a key pair and sign the image by digest.

```
cosign generate-key-pair
DIG=$(docker buildx imagetools inspect REG:5000/web:1.0.0 --format '{{json
.Manifest.Digest}}' | tr -d '"')
cosign sign --key cosign.key REG:5000/web@$DIG
```

- 4 Attach the SBOM as an attestation.

```
cosign attest --key cosign.key --type spdxjson --predicate web.sbom.json
REG:5000/web@$DIG
```

- 5 Verify — then try verifying an unsigned tag.

```
cosign verify --key cosign.pub REG:5000/web@$DIG | jq '[0].critical.identity'
cosign verify --key cosign.pub REG:5000/web:unsigned
```

EXPECT Second command: “no matching signatures”.

- 6 Pick one CRITICAL finding from step 1 and write the remediation: rebuild on a patched base, or justify accepting it.

CHECKPOINT

1. Why sign the digest and not the tag?
2. What does an SBOM let you answer in minutes when the next major CVE lands?

IF IT GOES WRONG

- cosign and self-signed registries: export SSL_CERT_FILE=~/.reg/certs/reg.crt.

Stretch	Write a one-line policy you would enforce: which severities block a release, and who may grant an exception.
----------------	--

Cleanup	<code>rm -f web.sbom.json</code>
----------------	----------------------------------

Running Containers

The runtime, storage, networking, multi-container apps, configuration, and observability on a single host and across two.

Lab 11 Lifecycle, Limits, and the OOM Kill

Prerequisites Lab 3.

STEPS

- 1 Walk the lifecycle and watch the state column.

```
docker create --name life alpine:3.20 sleep 300
docker ps -a --filter name=life
docker start life; docker pause life; docker ps --filter name=life
docker unpause life; docker stop life; docker rm life
```

- 2 Cap CPU and watch it hold.

```
docker run -d --name cpu --cpus 0.5 alpine:3.20 sh -c 'while ;; do ;; done'
docker stats --no-stream cpu
```

EXPECT CPU % hovers near 50%.

- 3 Drive a container past its memory limit.

```
docker run --name oom --memory 64m --memory-swap 64m alpine:3.20 sh -c 'tail
/dev/zero'
echo "exit: $?"
```

EXPECT exit: 137

- 4 Diagnose from state: the kernel, not the app, ended it.

```
docker inspect oom | jq '[0].State | {Status, ExitCode, OOMKilled}'
sudo dmesg | grep -i -m3 "killed process"
```

- 5 Watch a restart policy give up.

```
docker run -d --name flaky --restart on-failure:3 alpine:3.20 sh -c 'exit 1'
sleep 15; docker inspect -f '{{.RestartCount}} {{.State.Status}}' flaky
```

- 6 Make a container that ignores SIGTERM and time docker stop.

```
docker run -d --name stubborn alpine:3.20 sh -c 'trap "" TERM; sleep 600'
time docker stop stubborn
```

EXPECT ≈10 s, then SIGKILL (exit 137, OOMKilled false).

CHECKPOINT

1. Two containers exit 137. How do you tell which was OOM-killed?
2. Which runtime layer actually applies the memory limit?
3. Why did the shell ignore TERM when running as PID 1?

IF IT GOES WRONG

- If OOMKilled is false but exit is 137, something sent SIGKILL — check docker events for a kill or stop.

Stretch	Set --memory 64m but leave swap unlimited. Does tail /dev/zero still get OOM-killed? Explain what changed.
----------------	--

Cleanup	<code>docker rm -f cpu oom flaky stubborn</code>
----------------	--

Lab 12 Volumes, Bind Mounts, and the Permission Error

Prerequisites Lab 5.

STEPS

- 1 Write to a named volume, delete the container, read from a new one.

```
docker volume create notes
docker run --rm -v notes:/data alpine:3.20 sh -c 'date > /data/stamp'
docker run --rm -v notes:/data alpine:3.20 cat /data/stamp
docker volume inspect notes | jq '[0].Mountpoint'
```

- 2 Prove the writable layer does not persist.

```
docker run --name w alpine:3.20 sh -c 'echo gone > /stamp'; docker rm w
docker run --rm alpine:3.20 cat /stamp
```

- 3 Stage the fault: a root-owned host directory, a non-root container.

```
mkdir -p ~/labs/l12/data && sudo chown root:root ~/labs/l12/data && sudo chmod 755 ~/labs/l12/data
docker run --rm --user 1000:1000 -v ~/labs/l12/data:/data alpine:3.20 touch /data/x
```

EXPECT touch: /data/x: Permission denied

- 4 Diagnose: compare the container UID with the directory owner as the host sees it.

```
docker run --rm --user 1000:1000 alpine:3.20 id
ls -ln ~/labs/l12
```

- 5 Fix at the right layer (ownership), not by running as root.

```
sudo chown 1000:1000 ~/labs/l12/data
docker run --rm --user 1000:1000 -v ~/labs/l12/data:/data alpine:3.20 touch /data/x
&& echo fixed
```

- 6 Use tmpfs for data that must never touch disk.

```
docker run --rm --tmpfs /scratch:size=16m alpine:3.20 df -h /scratch
```

CHECKPOINT

1. When is a bind mount the right choice over a volume?
2. Why do permission errors on mounts behave differently in rootless mode?

IF IT GOES WRONG

- SELinux hosts (Fedora/RHEL): add :Z to the bind mount, or the error persists after chown.

Stretch	Share one volume between a writer and a reader container. What stops two writers from corrupting a file?
----------------	--

Cleanup	<code>docker volume rm notes; sudo rm -rf ~/labs/l12</code>
----------------	---

Lab 13 Ports, Bridges, and Name Resolution

Prerequisites Labs 2 and 5.

STEPS

- 1 Publish a port and read the mapping. Container port and host port are different numbers on purpose.

```
docker run -d --name web -p 8081:80 nginx:1.27
docker port web
curl -sI localhost:8081 | head -n1
```

- 2 See what the runtime built: a bridge, a veth pair, NAT rules.

```
ip -br addr show docker0
bridge link
sudo iptables -t nat -L DOCKER -n
```

- 3 Try name resolution on the default bridge (fails), then on a user-defined network (works).

```
docker run --rm alpine:3.20 ping -c1 web
docker network create appnet
docker network connect appnet web
docker run --rm --network appnet alpine:3.20 wget -qO- web | head -n 4
```

- 4 Inspect DNS inside the network.

```
docker run --rm --network appnet alpine:3.20 cat /etc/resolv.conf
```

EXPECT nameserver 127.0.0.11 (the runtime's embedded DNS)

- 5 Stage the fault: a server listening only on loopback inside its container.

```
docker run -d --name loop -p 8082:8000 python:3.12-slim python -m http.server 8000 -
-bind 127.0.0.1
curl -s localhost:8082 || echo "unreachable"
```

- 6 Diagnose from inside the network namespace, then fix the bind address.

```
PID=$(docker inspect -f '{{.State.Pid}}' loop)
sudo nsenter -t $PID -n ss -tln
docker rm -f loop
docker run -d --name loop -p 8082:8000 python:3.12-slim python -m http.server 8000 -
-bind 0.0.0.0
```

7 Try host networking and note that `-p` is ignored.

```
docker run --rm --network host alpine:3.20 ip -br addr
```

CHECKPOINT

1. Why did 127.0.0.1 inside the container not mean the host?
2. What error does a second container get publishing 8081 again?

IF IT GOES WRONG

- Firewall managers (ufw, firewalld) can interact with Docker's NAT rules; check both when a published port is unreachable from another machine.

Stretch	Publish a port bound only to the host's loopback (<code>-p 127.0.0.1:8083:80</code>) and test it from another machine.
----------------	--

Cleanup	<code>docker rm -f web loop; docker network rm appnet</code>
----------------	--

Lab 14 Isolated Networks and Crossing Hosts

Prerequisites Lab 13. Two hosts (A, B) with TCP 2377, TCP/UDP 7946, UDP 4789 open between them.

STEPS

- 1 Build a three-tier layout: web on frontend, db on backend, api on both.

```
docker network create frontend
docker network create --internal backend
docker run -d --name db --network backend redis:7
docker run -d --name api --network backend alpine:3.20 sleep 3600
docker network connect frontend api
docker run -d --name webx --network frontend alpine:3.20 sleep 3600
```

- 2 Confirm the isolation matrix.

```
docker exec api ping -c1 db
docker exec api ping -c1 webx
docker exec webx ping -c1 db
docker exec api ping -c1 1.1.1.1
```

EXPECT ok · ok · bad address · fails (backend is --internal, but api reaches out via frontend – check which)

- 3 Record the matrix as a 3×3 table of who can reach whom.
- 4 On host A, create a swarm and an attachable overlay.

```
docker swarm init --advertise-addr <A-IP>
docker network create -d overlay --attachable ov
```

- 5 On host B, join with the token printed on A, then run a container on each host.

```
# on B
docker swarm join --token <token> <A-IP>:2377
docker run -d --name b1 --network ov alpine:3.20 sleep 3600
# on A
docker run --rm --network ov alpine:3.20 ping -c3 b1
```

- 6 Break it: block UDP 4789 on B and retry. Note the symptom — names resolve but packets do not arrive.

```
sudo iptables -I INPUT -p udp --dport 4789 -j DROP
```

CHECKPOINT

1. Why does DNS still resolve b1 when the data path is blocked?
2. What does --internal do that a separate network alone does not?

IF IT GOES WRONG

- Cloud VMs: security groups must allow the swarm ports as well as the host firewall.

Stretch	Replace the overlay with plain routing: give each host's bridge a distinct subnet and add static routes. Compare the effort.
----------------	--

Cleanup	<pre>sudo iptables -D INPUT -p udp --dport 4789 -j DROP; docker rm -f db api webx b1; docker swarm leave --force; docker network prune -f</pre>
----------------	---

Lab 15 A Multi-Container App with Compose

Prerequisites Lab 13.

STEPS

- 1 Create the application file.

```
mkdir -p ~/labs/l15 && cd ~/labs/l15
cat > compose.yaml <<'EOF'
services:
  web:
    image: python:3.12-slim
    command: sh -c "pip install -q flask redis psycopg[binary] && flask --app app
run -h 0.0.0.0"
    working_dir: /app
    volumes: [./app.py:/app/app.py:ro]
    env_file: .env
    ports: ["8000-8002:5000"]
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }
  db:
    image: postgres:16
    env_file: .env
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 10
  cache:
    image: redis:7
volumes:
  pgdata:
EOF
```

- 2 Add configuration and the app.

```

printf 'POSTGRES_PASSWORD=labpass\nDB_HOST=db\n' > .env
cat > app.py <<'EOF'
import os, socket, redis, psycpg
from flask import Flask
app = Flask(__name__)
r = redis.Redis(host="cache")
@app.get("/")
def hit():
    psycpg.connect(host=os.environ["DB_HOST"], user="postgres",
password=os.environ["POSTGRES_PASSWORD"]).close()
    return f"{socket.gethostname()} hits={r.incr('hits')}\n"
EOF

```

- 3 Bring it up and watch the startup order.

```

docker compose up -d
docker compose ps
curl localhost:8000

```

- 4 Scale web to three instances and see requests land on different containers.

```

docker compose up -d --scale web=3
for p in 8000 8001 8002; do curl -s localhost:$p; done

```

- 5 Remove the healthcheck condition, run docker compose down && up, and capture the race in the logs.

```

docker compose logs web | grep -i -m3 error

```

CHECKPOINT

1. What does “service_started” guarantee, and what does it not?
2. Which parts of this file would you not use in production, and why?

IF IT GOES WRONG

- “port is already allocated”: another lab still holds 8000 — docker ps.

Stretch	Put an nginx reverse proxy in front of the three web instances so clients use a single port.
----------------	--

Cleanup	docker compose down -v
----------------	------------------------

Lab 16 Externalized Config and Runtime Secrets

Prerequisites Lab 15.

STEPS

- 1 Show why environment variables are a poor place for secrets.

```
docker run -d --name leaky -e DB_PASSWORD=hunter2 alpine:3.20 sleep 600
docker inspect leaky | jq '.[0].Config.Env'
sudo cat /proc/$(docker inspect -f '{{.State.Pid}}' leaky)/environ | tr '\0' '\n'
```

- 2 Create per-environment config files and a secret file.

```
mkdir -p ~/labs/l16/{dev,prod} && cd ~/labs/l16
echo 'log_level=debug' > dev/app.conf
echo 'log_level=warn' > prod/app.conf
printf 'hunter2' > db_password.txt && chmod 600 db_password.txt
```

- 3 Declare the secret in Compose; it arrives as a file under /run/secrets.

```
cat > compose.yaml <<'EOF'
services:
  app:
    image: alpine:3.20
    command: sh -c 'cat /etc/app/app.conf; ls -l /run/secrets; sleep 600'
    volumes: ["./${ENV:-dev}/app.conf:/etc/app/app.conf:ro"]
    secrets: [db_password]
secrets:
  db_password:
    file: ./db_password.txt
EOF
ENV=prod docker compose up -d && docker compose logs app
```

- 4 Prove the secret is absent from the image, the config, and the logs.

```
docker compose exec app cat /run/secrets/db_password; echo
docker inspect $(docker compose ps -q app) | grep -c hunter2
docker compose logs app | grep -c hunter2
docker history --no-trunc alpine:3.20 | grep -c hunter2
```

EXPECT hunter2

0

0

0

CHECKPOINT

1. Name three ways an environment-variable secret commonly leaks.
2. What would you change so prod reads config from somewhere other than the local disk?

IF IT GOES WRONG

- Setting “did not take”: check the mount path and that the app reads the file at startup, not only at build.

Stretch	Rotate the secret without rebuilding or re-creating the image. What had to restart?
----------------	---

Cleanup	<code>docker compose down; docker rm -f leaky; rm db_password.txt</code>
----------------	--

Lab 17 Logs, Health Checks, and Evidence That Survives

Prerequisites Lab 11.

STEPS

- 1 Run with bounded logs and find where they live.

```
docker run -d --name logged --log-opt max-size=1m --log-opt max-file=3 nginx:1.27
docker inspect -f '{{.LogPath}}' logged
```

- 2 Add a health check that tests real work, not just liveness.

```
docker run -d --name healthy --health-cmd 'curl -fs localhost/ || exit 1' --health-
interval 5s nginx:1.27
sleep 12; docker inspect healthy | jq '.[0].State.Health.Status, .
[0].State.Health.Log[-1]'
```

- 3 Make the health check fail while the process keeps running.

```
docker exec healthy rm /usr/share/nginx/html/index.html
sleep 20; docker ps --filter name=healthy
```

EXPECT STATUS ... (unhealthy) – running, but not working.

- 4 Start an event recorder in a second terminal.

```
docker events --format '{{json .}}' > ~/events.jsonl
```

- 5 Run a self-deleting container that crashes, sending logs to journald so they outlive it.

```
docker run --rm --name ghost --log-driver journald alpine:3.20 sh -c 'echo starting;
echo fatal: config missing >&2; exit 3'
docker logs ghost
```

EXPECT Error: No such container: ghost

- 6 Recover the evidence.

```
journalctl CONTAINER_NAME=ghost --no-pager
grep ghost ~/events.jsonl | jq -r '.status + " " + (.Actor.Attributes.exitCode //
"'')
```

- 7 Compare per-container metrics.

```
docker stats --no-stream
```

CHECKPOINT

1. Why is “running” a weaker claim than “healthy”?
2. Which evidence would have been lost without steps 4 and 5?

IF IT GOES WRONG

- Rootless Docker may lack journald access; use --log-driver local and skip --rm instead.

Stretch	Ship logs from two containers to a single Loki or Elasticsearch instance and query both at once.
----------------	--

Cleanup	<code>docker rm -f logged healthy; rm ~/events.jsonl</code>
----------------	---

Kubernetes

From one host to a cluster: the reconciliation model, the components, and the objects that run, connect, store, and scale workloads.

Lab 18 Your First Cluster

Prerequisites Lab 3 host with 8 GB RAM. Install kubectl and kind.

STEPS

- 1 Define a cluster with one control-plane node and one worker.

```
mkdir -p ~/labs/k8s && cd ~/labs/k8s
cat > kind.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - { containerPort: 80, hostPort: 80 }
  labels: { ingress-ready: "true" }
- role: worker
EOF
kind create cluster --name lab --config kind.yaml
```

- 2 Confirm both nodes are Ready.

```
kubectl get nodes -o wide
```

```
EXPECT lab-control-plane    Ready    control-plane
        lab-worker          Ready    <none>
```

- 3 Check the control plane's own health endpoints.

```
kubectl get --raw='/readyz?verbose' | tail -n 5
kubectl -n kube-system get pods
```

- 4 Declare desired state and watch reconciliation create it.

```
kubectl create deployment hello --image=nginx:1.27 --replicas=2
kubectl get pods -w
```

- 5 Notice each “node” is itself a container on your host.

```
docker ps --filter name=lab-
```

CHECKPOINT

1. You asked for 2 replicas. What will happen if one pod is deleted, and which component notices?
2. Why is a kind cluster fine for learning but not for production?

IF IT GOES WRONG

- Pods stuck ContainerCreating: check host memory and inotify limits (sysctl fs.inotify.max_user_instances=512).

Stretch	Recreate the cluster with minikube instead of kind and list three differences you notice.
----------------	---

Cleanup	<code>kubectl delete deployment hello</code> <code># keep the cluster for Labs 19-23</code>
----------------	---

Lab 19 Components and Self-Healing

Prerequisites Lab 18 cluster.

STEPS

- 1 Map the control-plane components to their pods.

```
kubectl -n kube-system get pods -o wide
docker exec lab-control-plane ls /etc/kubernetes/manifests
```

- 2 Inspect the node agent and the runtime underneath — the same stack as Part II.

```
docker exec lab-worker systemctl status kubelet --no-pager | head -n 5
docker exec lab-worker crictl ps
```

- 3 Deploy something and kill its container directly on the node.

```
kubectl create deployment heal --image=nginx:1.27
CID=$(docker exec lab-worker crictl ps --name nginx -q | head -n1)
docker exec lab-worker crictl rm -f $CID
kubectl get pods -l app=heal -w
```

EXPECT **RESTARTS** increments: the kubelet restarted it.

- 4 Delete the pod itself and watch a controller replace it with a new name.

```
kubectl delete pod -l app=heal
kubectl get events --sort-by=.lastTimestamp | tail -n 8
```

- 5 Stop the scheduler and create a pod.

```
docker exec lab-control-plane mv /etc/kubernetes/manifests/kube-scheduler.yaml
/root/
kubectl run orphan --image=nginx:1.27
kubectl get pod orphan
```

EXPECT **Pending** — no scheduler to assign a node.

- 6 Restore the scheduler and watch the pending pod get placed.

```
docker exec lab-control-plane mv /root/kube-scheduler.yaml
/etc/kubernetes/manifests/
kubectl get pod orphan -w
```

CHECKPOINT

1. What was the difference in who healed step 3 versus step 4?
2. What happens to running pods if the API server is down? To new deployments?

IF IT GOES WRONG

- If the scheduler pod does not return, check that the manifest was moved back with the same filename.

Stretch	Stop the kubelet on lab-worker. How long until the node is NotReady, and what happens to its pods after that?
----------------	---

Cleanup	<code>kubectl delete deployment heal; kubectl delete pod orphan</code>
----------------	--

Lab 20 Rollouts, Rollbacks, and a Crash Loop

Prerequisites Lab 18 cluster.

STEPS

1 Deploy with probes.

```
cat > web.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - name: nginx
        image: nginx:1.26
        readinessProbe: { httpGet: { path: /, port: 80 }, periodSeconds: 3 }
        livenessProbe: { httpGet: { path: /, port: 80 }, periodSeconds: 10 }
EOF
kubectl apply -f web.yaml && kubectl rollout status deploy/web
```

2 Roll forward and watch ReplicaSets swap.

```
kubectl set image deploy/web nginx=nginx:1.27
kubectl rollout status deploy/web
kubectl get rs -l app=web
kubectl rollout history deploy/web
```

3 Roll back.

```
kubectl rollout undo deploy/web
kubectl get deploy web -o jsonpath='{.spec.template.spec.containers[0].image}'; echo
```

4 Induce a crash loop with a missing required setting.

```
kubectl create deployment crashy --image=busybox:1.36 -- sh -c 'test -n "$API_URL"
|| { echo "API_URL not set" >&2; exit 2; }; sleep 3600'
kubectl get pods -l app=crashy -w
```

EXPECT Error → CrashLoopBackOff, RESTARTS climbing

5 Diagnose: exit code, previous logs, events.

```
kubectl describe pod -l app=crashy | grep -A5 "Last State"
kubectl logs -l app=crashy --previous
```

6 Fix it declaratively.

```
kubectl set env deploy/crashy API_URL=http://api
kubectl get pods -l app=crashy
```

CHECKPOINT

1. Is this crash loop an application bug, a config problem, or a dependency problem? What evidence says so?
2. What would a failing readiness probe do differently from a failing liveness probe?

IF IT GOES WRONG

- A rollout that stalls usually has a readiness probe that never passes: `kubectl describe` the new pods.

Stretch	Change the rollout strategy to <code>maxUnavailable: 0</code> , <code>maxSurge: 1</code> and time a rollout. What did you trade?
----------------	--

Cleanup	<code>kubectl delete deploy crashy</code> <i># keep web for Lab 21</i>
----------------	--

Lab 21 Services, DNS, and Ingress

Prerequisites Lab 20 “web” deployment.

STEPS

- 1 Expose the deployment and check its endpoints.

```
kubectl expose deploy web --port 80
kubectl get svc web
kubectl get endpointslices -l kubernetes.io/service-name=web
```

- 2 Reach it by name from another pod.

```
kubectl run -it --rm probe --image=busybox:1.36 --restart=Never -- sh -c 'nslookup
web; wget -qO- web | head -n 4'
```

- 3 Install an ingress controller for kind.

```
kubectl apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-ingress-nginx.yaml
kubectl -n ingress-nginx wait --for=condition=ready pod -l
app.kubernetes.io/component=controller --timeout=120s
```

- 4 Route a host and path to the service.

```
kubectl create ingress web --class=nginx --rule="web.lab.local/*=web:80"
curl -s -H 'Host: web.lab.local' localhost | head -n 4
```

- 5 Stage the fault: a service whose selector matches nothing.

```
kubectl create service clusterip broken --tcp=80:80
kubectl get endpointslices -l kubernetes.io/service-name=broken
```

- 6 Diagnose by comparing the selector with pod labels, then fix.

```
kubectl get svc broken -o jsonpath='{.spec.selector}'; echo
kubectl get pods --show-labels | head -n 4
kubectl patch svc broken -p '{"spec":{"selector":{"app":"web"}}}'
```

CHECKPOINT

1. Why do pods need a service at all if every pod has its own IP?
2. Which service type would you use to reach this from outside a cloud cluster without an ingress?

IF IT GOES WRONG

- Endpoints exist but traffic fails: check the targetPort matches the container port, and that pods are Ready.

Stretch	Add a second path rule that routes /api to a different service. Test both paths with curl.
----------------	--

Cleanup	<code>kubectl delete ingress web; kubectl delete svc web broken; kubectl delete deploy web</code>
----------------	---

Lab 22 Persistent Storage, ConfigMaps, and Secrets

Prerequisites Lab 18 cluster (kind ships a default StorageClass).

STEPS

- 1 Look at the storage tiers on offer.

```
kubectl get storageclass
```

- 2 Create configuration and a secret.

```
kubectl create configmap app-config --from-literal=GREETING=hello-from-config
kubectl create secret generic app-secret --from-literal=API_KEY=lab22-key
```

- 3 Deploy a StatefulSet with a volume claim template and both injected.

```
cat > sts.yaml <<'EOF'
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: store }
spec:
  serviceName: store
  replicas: 2
  selector: { matchLabels: { app: store } }
  template:
    metadata: { labels: { app: store } }
    spec:
      containers:
      - name: app
        image: busybox:1.36
        command: ["sh", "-c", "echo $(hostname) $(date) >> /data/log; sleep 3600"]
        envFrom: [{ configMapRef: { name: app-config } }]
        volumeMounts:
        - { name: data, mountPath: /data }
        - { name: secret, mountPath: /etc/secret, readOnly: true }
      volumes:
      - name: secret
        secret: { secretName: app-secret }
      volumeClaimTemplates:
      - metadata: { name: data }
        spec: { accessModes: [ReadWriteOnce], resources: { requests: { storage: 100Mi } } }
    } }
EOF
kubectl apply -f sts.yaml && kubectl get pvc -w
```

4 Verify config and secret arrived.

```
kubectl exec store-0 -- sh -c 'echo $GREETING; cat /etc/secret/API_KEY'
```

5 Delete a pod and prove identity and data survive.

```
kubectl delete pod store-0
kubectl wait --for=condition=ready pod/store-0
kubectl exec store-0 -- cat /data/log
```

EXPECT Two lines, both from store-0.

6 See the limit of built-in secrets.

```
kubectl get secret app-secret -o jsonpath='{.data.API_KEY}' | base64 -d; echo
```

7 Update the ConfigMap. Check the env var (does not change) versus a mounted file (eventually does).

CHECKPOINT

1. Why is base64 not a protection? What actually protects a Secret?
2. What happens to the PVCs if you delete the StatefulSet?

IF IT GOES WRONG

- PVC Pending: `kubectl describe pvc` — look for a missing StorageClass or an unsupported access mode.

Stretch

Mount the ConfigMap as a file instead of env vars and time how long an update takes to appear.

Cleanup

```
kubectl delete sts store; kubectl delete pvc -l app=store; kubectl delete
cm app-config; kubectl delete secret app-secret
```

Lab 23 Requests, Autoscaling, and Draining

Prerequisites Lab 18 cluster.

STEPS

- 1 Install metrics-server (kind needs the insecure-TLS flag).

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl -n kube-system patch deploy metrics-server --type=json -p ' [{ "op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls" } ] '
kubectl top nodes
```

- 2 Deploy with requests and limits.

```
kubectl create deployment php --image=registry.k8s.io/hpa-example
kubectl set resources deploy php --requests=cpu=200m,memory=64Mi --limits=cpu=500m,memory=128Mi
kubectl expose deploy php --port 80
```

- 3 Create the autoscaler.

```
kubectl autoscale deploy php --cpu-percent=50 --min=1 --max=8
kubectl get hpa php
```

- 4 Drive load in a second terminal and watch replicas climb.

```
kubectl run load --image=busybox:1.36 --restart=Never -- sh -c 'while true; do wget -qO- http://php >/dev/null; done'
kubectl get hpa php -w
```

- 5 Stop the load and note how long scale-in takes compared with scale-out.

```
kubectl delete pod load
```

- 6 Protect availability, then drain the worker.

```
kubectl create pdb php --selector=app=php --min-available=1
kubectl drain lab-worker --ignore-daemonsets --delete-emptydir-data
kubectl get pods -o wide
kubectl uncordon lab-worker
```

CHECKPOINT

1. Why does the autoscaler need requests to be set?
2. Which pods are placed first when a node is short on memory — and which are evicted first?

IF IT GOES WRONG

- HPA shows <unknown>: metrics-server is not ready or the pods have no CPU request.

Stretch

Add a memory-based metric to the autoscaler. Why is memory usually a worse scaling signal than CPU?

Cleanup

```
kubectl delete hpa php; kubectl delete pdb php; kubectl delete deploy php;  
kubectl delete svc php
```

Lab 24 Managed Cluster, Helm, and GitOps

Prerequisites A cloud account with a small managed cluster (EKS, GKE, or AKS) — or your kind cluster as a no-cost substitute. Install helm. A Git repository you can push to.

STEPS

- 1 Point kubectl at the managed cluster and note what you no longer see.

```
kubectl config get-contexts
kubectl get nodes
kubectl -n kube-system get pods
```

EXPECT No API server, scheduler, or etcd pods: the provider runs the control plane.

- 2 Scaffold a chart and commit it with per-environment values.

```
cd ~/labs && git clone <your-repo> gitops && cd gitops
helm create hello
printf 'replicaCount: 2\nimage:\n tag: "1.27"\n' > values-prod.yaml
git add . && git commit -m "hello chart" && git push
```

- 3 Install Argo CD.

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

- 4 Declare the application — the cluster now pulls its desired state from Git.

```
cat > app.yaml <<'EOF'
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: hello, namespace: argocd }
spec:
  project: default
  source:
    repoURL: <your-repo-url>
    path: hello
    helm: { valueFiles: [../values-prod.yaml] }
  destination: { server: https://kubernetes.default.svc, namespace: hello }
  syncPolicy: { automated: { selfHeal: true, prune: true }, syncOptions:
[CreateNamespace=true] }
EOF
kubectl apply -f app.yaml
kubectl -n hello get pods -w
```

- 5 Change replicaCount to 3 in Git, push, and watch the cluster follow.
- 6 Create drift by hand and watch it get corrected.

```
kubectl -n hello scale deploy --all --replicas=1
kubectl -n hello get deploy -w
```

CHECKPOINT

1. What did the managed service take off your plate, and what is still yours?
2. Your hand-edit was reverted. Why is that a feature?

IF IT GOES WRONG

- Argo CD shows OutOfSync but will not apply: check the repo URL is reachable from the cluster and the chart path is correct.

Stretch	Add a staging environment as a second Application with its own values file, driven from the same chart.
----------------	---

Cleanup	<code>kubectl delete -f app.yaml; kubectl delete ns argocd hello</code> # then delete the cloud cluster to stop charges
----------------	---

Production Concerns

Security, delivery pipelines, stateful workloads, and the wider cloud-native ecosystem.

Lab 25 Hardening a Container

Prerequisites Labs 2 and 18.

STEPS

- 1 Record the default privileges.

```
docker run --rm alpine:3.20 sh -c 'id; grep CapEff /proc/self/status'
```

- 2 Run hardened.

```
docker run -d --name hard --user 10001:10001 --cap-drop ALL --read-only --tmpfs /tmp
--security-opt no-new-privileges --pids-limit 64 alpine:3.20 sleep 3600
docker exec hard sh -c 'id; grep CapEff /proc/self/status'
```

EXPECT uid=10001 ... CapEff: 0000000000000000

- 3 Prove each restriction.

```
docker exec hard touch /etc/x
docker exec hard chown 0 /tmp
docker exec hard ping -c1 1.1.1.1
docker exec hard sh -c 'touch /tmp/ok && echo tmp-writable'
```

EXPECT Read-only file system · Operation not permitted · permission denied · tmp-writable

- 4 Add a seccomp profile that denies one syscall and prove it.

```
curl -sO
https://raw.githubusercontent.com/moby/moby/master/profiles/seccomp/default.json
jq '.syscalls += [{"names":["mkdir","mkdirat"],"action":"SCMP_ACT_ERRNO"}]'
default.json > deny-mkdir.json
docker run --rm --security-opt seccomp=deny-mkdir.json alpine:3.20 mkdir /tmp/x
```

- 5 Enforce the same posture cluster-wide with Pod Security Admission.

```
kubectl create ns locked
kubectl label ns locked pod-security.kubernetes.io/enforce=restricted
kubectl -n locked run bad --image=nginx:1.27
```

EXPECT Error ... violates PodSecurity "restricted:latest"

- 6 Write a pod spec that passes: runAsNonRoot, allowPrivilegeEscalation: false, capabilities.drop [ALL], seccompProfile RuntimeDefault. Apply it to “locked”.

CHECKPOINT

1. Which single hardening step gives the most protection against a container escape, and why?
2. Which features from Lab 2 did each restriction rely on?

IF IT GOES WRONG

- Apps that need port 80 as non-root: add back only NET_BIND_SERVICE, or listen on 8080.

Stretch	Run docker-bench-security (or kube-bench) against your lab and fix the top three findings.
----------------	--

Cleanup	<code>docker rm -f hard; kubectl delete ns locked</code>
----------------	--

Lab 26 A Pipeline from Commit to Cluster

Prerequisites Labs 10 and 24. A GitHub repository with Actions enabled (the same stages apply to any CI system).

STEPS

- 1 Add the Lab 7 app and Dockerfile to the repository.
- 2 Write the workflow.

```
mkdir -p .github/workflows
cat > .github/workflows/ship.yaml <<'EOF'
name: ship
on: { push: { branches: [main] } }
permissions: { contents: write, packages: write, id-token: write }
jobs:
  ship:
    runs-on: ubuntu-latest
    env: { IMAGE: ghcr.io/${{ github.repository }}/app }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${{ github.actor }}, password: ${{
secrets.GITHUB_TOKEN }} }
      - run: docker build -t $IMAGE:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@0.28.0
        with: { image-ref: "${{ env.IMAGE }}:${{ github.sha }}", severity: CRITICAL,
exit-code: "1" }
      - run: docker push $IMAGE:${{ github.sha }}
      - uses: sigstore/cosign-installer@v3
      - run: cosign sign --yes $IMAGE:${{ github.sha }}
      - name: promote via GitOps
        run: |
          sed -i "s/tag: ./tag: \"${{ github.sha }}\"/" values-prod.yaml
          git config user.name ci && git config user.email ci@example.com
          git commit -am "deploy ${{ github.sha }}" && git push
EOF
```

- 3 Point the Lab 24 Argo CD application at this repository and chart.
- 4 Commit a change and follow it through each stage to a running pod with the new SHA.

```
kubectl -n hello get deploy -o jsonpath='{..image}'; echo
```

- 5 Introduce a known-vulnerable base image and confirm the scan gate stops the pipeline before push.

CHECKPOINT

1. Where is the handoff between build and deploy in this pipeline?
2. Why is the image tagged with the commit SHA rather than a branch name?

IF IT GOES WRONG

- Works locally, fails in CI: compare architectures (CI is amd64; your laptop may be arm64) and look for files ignored by .gitignore but used by the build.

Stretch	Add a gate that verifies the cosign signature before Argo CD syncs, using a policy engine such as Kyverno.
----------------	--

Cleanup	Disable the workflow or delete the branch when finished.
----------------	--

Lab 27 A Stateful Database: Backup and Restore

Prerequisites Lab 22.

STEPS

- 1 Deploy PostgreSQL as a StatefulSet with a headless service.

```
kubectl create secret generic pg --from-literal=POSTGRES_PASSWORD=labpass
cat > pg.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata: { name: pg }
spec: { clusterIP: None, selector: { app: pg }, ports: [{ port: 5432 }] }
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: pg }
spec:
  serviceName: pg
  replicas: 1
  selector: { matchLabels: { app: pg } }
  template:
    metadata: { labels: { app: pg } }
    spec:
      containers:
        - name: postgres
          image: postgres:16
          envFrom: [{ secretRef: { name: pg } }]
          env: [{ name: PGDATA, value: /var/lib/postgresql/data/pgdata }]
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
      volumeClaimTemplates:
        - metadata: { name: data }
          spec: { accessModes: [ReadWriteOnce], resources: { requests: { storage: 1Gi } } }
    }
EOF
kubectl apply -f pg.yaml && kubectl wait --for=condition=ready pod/pg-0 --
timeout=120s
```

- 2 Write data and note the stable DNS name.

```
kubectl exec pg-0 -- psql -U postgres -c "create table orders(id int); insert into
orders select generate_series(1,1000);"
kubectl run -it --rm dns --image=busybox:1.36 --restart=Never -- nslookup pg-0.pg
```

- 3 Back up to your workstation.

```
kubectl exec pg-0 -- pg_dump -U postgres -Fc postgres > pg.dump
ls -lh pg.dump
```

4 Simulate the incident.

```
kubectl exec pg-0 -- psql -U postgres -c "drop table orders;"
```

5 Restore and verify.

```
kubectl exec -i pg-0 -- pg_restore -U postgres -d postgres < pg.dump
kubectl exec pg-0 -- psql -U postgres -tc "select count(*) from orders;"
```

EXPECT 1000

6 Delete the pod and confirm pg-0 comes back with the same claim and data.

7 Write down your recovery time and what an operator (such as CloudNativePG) would automate.

CHECKPOINT

1. What makes a database a poor fit for a Deployment?
2. Is a backup stored inside the same cluster a real backup?

IF IT GOES WRONG

- “directory not empty” on first start: PGDATA must be a subdirectory of the mount — which is why it is set above.

Stretch	Replace the StatefulSet with the CloudNativePG operator and schedule an automatic backup.
----------------	---

Cleanup	<code>kubectl delete -f pg.yaml; kubectl delete pvc data-pg-0; kubectl delete secret pg</code>
----------------	--

Lab 28 Adding a Service Mesh

Prerequisites Lab 18 cluster. Install the linkerd CLI. (Alternative: deploy the same app to a serverless container platform such as Knative or Cloud Run.)

STEPS

- 1 Deploy a two-service app to mesh.

```
kubectl create ns mesh
kubectl -n mesh create deployment web --image=nginx:1.27
kubectl -n mesh expose deploy web --port 80
kubectl -n mesh create deployment client --image=curlimages/curl -- sh -c 'while
true; do curl -s web >/dev/null; sleep 1; done'
```

- 2 Install the mesh and its dashboard extension.

```
linkerd check --pre
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -
linkerd viz install | kubectl apply -f -
linkerd check
```

- 3 Inject sidecars — note the pods go from 1/1 to 2/2 containers.

```
kubectl -n mesh get deploy -o yaml | linkerd inject - | kubectl apply -f -
kubectl -n mesh get pods
```

- 4 Read the metrics you got for free.

```
linkerd viz -n mesh stat deploy
linkerd viz -n mesh top deploy/web
```

- 5 Confirm traffic between the services is now mutually authenticated.

```
linkerd viz -n mesh edges deploy
```

- 6 Write one paragraph: what the mesh took off the application, and what it added to operate.

CHECKPOINT

1. Which pattern from Lab 20 is the mesh built on?
2. When would a mesh be overkill for this course's sample apps?

IF IT GOES WRONG

- linkerd check fails on certificates: re-run install; kind clusters sometimes need a fresh cluster.

Stretch	Deploy the same app to Knative Serving and watch it scale to zero after idle.
----------------	---

Cleanup	<pre>kubectl delete ns mesh; linkerd viz uninstall kubectl delete -f -; linkerd uninstall kubectl delete -f -</pre>
----------------	---

Troubleshooting and Next Steps

The full diagnostic framework applied to staged faults at every layer, and a plan for what comes after the course.

Lab 29 The Full Framework on a Multi-Layer Fault

Prerequisites Labs 4, 12, 20, 22. Ideally a partner stages the fault so you do not read the manifest.

SCENARIO

Partner: apply the manifest below. Learner: you are told only “the reports service went down after the last change”.

STEPS

- 1 Stage the fault (partner only).

```
cat > fault29.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata: { name: reports }
spec:
  selector: { matchLabels: { app: reports } }
  template:
    metadata: { labels: { app: reports } }
    spec:
      securityContext: { runAsUser: 10001 }
      containers:
      - name: app
        image: busybox:1.36
        command: ["sh","-c","echo start; touch /data/cache/ok && head -c 200m
/dev/zero | tail"]
        resources: { limits: { memory: 64Mi } }
        volumeMounts: [{ name: data, mountPath: /data }]
        volumes: [{ name: data, hostPath: { path: /srv/reports, type:
DirectoryOrCreate } }]
EOF
kubectl apply -f fault29.yaml
```

- 2 Define the problem and capture evidence before touching anything.

```
kubectl get pods -l app=reports -o wide
kubectl describe pod -l app=reports > evidence-describe.txt
kubectl logs -l app=reports --previous > evidence-logs.txt
kubectl get events --sort-by=.lastTimestamp > evidence-events.txt
```

- 3 Walk the layer map — application, container, image, runtime, host, orchestrator — writing one line of evidence for or against each.

- 4 Fix the first cause with one change. Re-evaluate: did the symptom change? If so, a second cause is now exposed.
- 5 Repeat until healthy. Expect two independent causes.
- 6 Write the root-cause report: timeline, each cause and its layer, the evidence that proved it, the fix, and a prevention step.

CHECKPOINT

1. Which cause was hidden by the other until the first was fixed?
2. Which evidence file would have been empty if you had deleted the pod first?

IF IT GOES WRONG

- Stuck? Re-read the events file line by line in time order before changing anything else.

Stretch	Write your own two-cause fault and swap with a partner. Score each other on time to first correct layer.
----------------	--

Cleanup	<code>kubectl delete -f fault29.yaml</code>
----------------	---

Lab 30 Staged Single-Container Faults

Prerequisites Part III. Stage each fault, then diagnose it without re-reading the staging command.

STEPS

- 1 Fault A — exits immediately.

```
docker run -d --name fa alpine:3.20 /app/start.sh
```

EXPECT Record: exit code, last log line, layer.

- 2 Fault B — crash loop.

```
docker run -d --name fb --restart always -e PORT=abc python:3.12-slim python -c
'import os; int(os.environ["PORT"])
```

- 3 Fault C — permission denied.

```
sudo mkdir -p /srv/fc && sudo chmod 700 /srv/fc
docker run -d --name fc --user 1000 -v /srv/fc:/out alpine:3.20 sh -c 'date >
/out/t'
```

- 4 Fault D — dies under load.

```
docker run -d --name fd --memory 32m python:3.12-slim python -c 'x =
bytearray(100_000_000)'
```

- 5 Fault E — image will not build.

```
mkdir -p ~/labs/l30 && cd ~/labs/l30
printf 'FROM alpine:3.20\nCOPY config.yaml /etc/app/\n' > Dockerfile
docker build -t fe .
```

- 6 For each fault, fill one row of the table: symptom, exit code, key evidence, layer, root cause, fix.

```
docker inspect <name> | jq '[0].State'
docker logs <name>
```

CHECKPOINT

1. Two faults exited with the same code. Which, and how did you tell them apart?
2. Which fault was the kernel's decision rather than the application's?

IF IT GOES WRONG

- Use `docker inspect` before `docker logs` — the exit code tells you where to look.

Stretch	Add a sixth fault of your own design that looks like a networking problem but is not.
----------------	---

Cleanup	<code>docker rm -f fa fb fc fd; sudo rm -rf /srv/fc ~/labs/l30</code>
----------------	---

Lab 31 Staged Cluster Faults

Prerequisites Part IV. Lab 18 cluster.

STEPS

1 Fault 1 — pending pod.

```
kubectl run f1 --image=nginx:1.27 --overrides='{ "spec": { "containers": [ { "name": "f1", "image": "nginx:1.27", "resources": { "requests": { "cpu": "64" } } } ] } }'
```

EXPECT Read: `kubectl describe pod f1` → Events

2 Fault 2 — image pull failure.

```
kubectl create deployment f2 --image=nginx:1.99-doesnotexist
```

3 Fault 3 — service with no endpoints.

```
kubectl create deployment f3 --image=nginx:1.27
kubectl create service clusterip f3svc --tcp=80:8080
```

4 Fault 4 — claim stuck pending.

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: f4 }
spec: { storageClassName: fast-ssd, accessModes: [ReadWriteOnce], resources: {
requests: { storage: 1Gi } } }
EOF
```

5 Fault 5 — DNS failure.

```
kubectl -n kube-system scale deploy coredns --replicas=0
kubectl run -it --rm f5 --image=busybox:1.36 --restart=Never -- nslookup
kubernetes.default
```

6 Read the whole cluster timeline, then fix each fault and record symptom, evidence, component, root cause, fix.

```
kubectl get events -A --sort-by=.lastTimestamp | tail -n 30
```

CHECKPOINT

1. Fault 3 has two separate problems. What are they?
2. Which fault would look identical to a network-plugin failure until you checked one thing?

IF IT GOES WRONG

- Always start with `kubectl describe`: the Events section usually names the cause.

Stretch	Break the network plugin itself (delete the kindnet DaemonSet) and describe how it differs from Fault 5.
----------------	--

Cleanup	<code>kubectl delete pod f1; kubectl delete deploy f2 f3; kubectl delete svc f3svc; kubectl delete pvc f4; kubectl -n kube-system scale deploy coredns --replicas=2</code>
----------------	--

Lab 32 Skills Assessment and Your 90-Day Plan

Prerequisites All previous labs, and your notes from them.

STEPS

- 1 Rate yourself 1–4 on each area (1 = followed the steps, 4 = could teach it): kernel features, images and builds, supply chain, runtime and storage, networking, Kubernetes objects, operations, security, CI/CD, troubleshooting.
- 2 For every area rated 1 or 2, pick the one lab you would repeat without the guide.
- 3 Choose a direction: platform engineering, cloud-native development, container security, or site reliability. Write two sentences on why.
- 4 Write the plan in three 30-day blocks. Each block gets one skill goal, one hands-on project on your home lab, and one measurable outcome (a certification exam, a published write-up, a working cluster).
- 5 Decide what you will keep running at home — for example a persistent kind or k3s cluster with GitOps from Lab 24 — so practice has somewhere to happen.
- 6 Choose at most three sources to follow for ecosystem news, and one thing you will deliberately ignore for now.

CHECKPOINT

1. Which concepts from Video 2 appeared again in the security and troubleshooting labs?
2. What would you need to show an employer to prove the specialization you chose?

IF IT GOES WRONG

- A plan with more than one goal per block is a wish list. Cut until it fits.

Stretch Book the date for your first exam or first public write-up and add it to the plan.

Cleanup `kind delete cluster --name lab` # when you no longer need it

Command cheat sheet

Command	What it tells you
Containers	
<code>docker ps -a</code>	Every container, including stopped ones, with status and exit code
<code>docker inspect NAME jq '[0].State'</code>	Exit code, OOMKilled, start and finish times, health
<code>docker logs --tail 50 -f NAME</code>	What the process wrote to stdout and stderr
<code>docker events --since 30m</code>	The timeline: create, start, die, kill, oom
<code>docker exec -it NAME sh</code>	A shell inside a running container
<code>docker stats --no-stream</code>	CPU, memory, network, and I/O per container
<code>docker inspect -f '{{.State.Pid}}' NAME</code>	The container's process ID on the host
<code>sudo nsenter -t PID -n ss -tln</code>	Run a host tool inside the container's network namespace
Images	
<code>docker history --no-trunc IMAGE</code>	Each layer and the instruction that made it
<code>docker image inspect IMAGE</code>	Digest, user, entrypoint, env, labels
<code>docker system df -v</code>	What is using disk space
<code>trivy image IMAGE</code>	Known vulnerabilities in the image's packages
Host	
<code>ps -ef --forest</code>	Process tree, including containerized processes
<code>ls -ln PATH</code>	Numeric owner and permissions — compare with the container's UID
<code>sudo ss -tlnp</code>	Which process is listening on which port
<code>sudo dmesg grep -i oom</code>	Kernel out-of-memory kills
<code>journalctl -u docker --since "10 min ago"</code>	The engine's own log
Kubernetes	

Command	What it tells you
<code>kubectl get pods -o wide</code>	Status, restarts, node, and pod IP
<code>kubectl describe pod NAME</code>	Configuration, last state, and the Events section — start here
<code>kubectl logs NAME --previous</code>	Logs from the container that just crashed
<code>kubectl get events -A --sort-by=.lastTimestamp</code>	The cluster timeline
<code>kubectl get endpointslices</code>	Which pods a service is actually sending traffic to
<code>kubectl rollout status history undo deploy/NAME</code>	Follow, review, or reverse a rollout
<code>kubectl top pods</code>	Live CPU and memory (needs metrics-server)
<code>kubectl run -it --rm dbg --image=busybox:1.36 --restart=Never --sh</code>	A throwaway shell inside the cluster network

Exit codes and signals

An exit code above 128 means the process was ended by a signal: subtract 128 to get the signal number.

Code	Meaning	Look first at
0	Finished normally	Was it supposed to keep running? Check the command.
1–2	Application error	Logs; configuration and environment
125	The engine could not run the container	The docker run flags themselves
126	Command found but not executable	File permissions in the image
127	Command not found	Entrypoint path; missing shell in minimal images
137	SIGKILL (9)	OOMKilled in inspect; otherwise a stop timeout or manual kill
139	SIGSEGV (11) — segmentation fault	Architecture mismatch, native library bug
143	SIGTERM (15) — asked to stop, and did	Who stopped it: docker events or kubectl get events

Signal	No.	In container work
SIGHUP	1	Many servers reload configuration
SIGINT	2	Ctrl+C in an attached terminal
SIGKILL	9	Cannot be caught. Sent after the stop grace period, or by the OOM killer
SIGTERM	15	docker stop and pod deletion send this first. PID 1 ignores it unless it installs a handler

Troubleshooting checklist

Copy this page for every fault in Labs 4 and 29–31. Fill it in before you change anything.

Problem statement	What fails, where, since when, what still works	
Evidence captured	Exit code · logs · events · inspect/describe output — saved to files before cleanup	
Layer	Typical failures	Evidence for / against
Application	Bug, missing setting, unreachable dependency	
Container	Wrong user, mount, port, env, limits	
Image	Moved tag, missing file, wrong architecture	
Runtime	OOM kill, restart policy, stop timeout	
Host	Permissions, disk, firewall, kernel	
Orchestrator	Scheduling, probes, services, DNS, storage claims	
Layer to test first	And why — the cheapest to rule out wins ties	
One change	Exactly one. Note what result would confirm or reject it	
Result	Did the symptom change? If so, is a second cause now visible?	
Root cause	Layer and cause, with the evidence that proves it	

Prevention

The check, test, or policy that stops it recurring
